

```

#![windows_subsystem = "windows"]
use serde::{Deserialize, Serialize};
use slint::{ComponentHandle, ModelRc, VecModel, Weak};
use std::{cell::RefCell, rc::Rc, thread, time::Duration};
slint::include_modules!();
#![link(name = "user32")]
extern "system" {
    fn FindWindowW(lp_c: *const u16, lp_w: *const u16) -> *mut std::ffi::c_void;
    fn ShowWindow(hwnd: *mut std::ffi::c_void, n_cmd_show: i32) -> i32;
    fn SetWindowPos(
        hwnd: *mut std::ffi::c_void,
        h_after: *mut std::ffi::c_void,
        x: i32,
        y: i32,
        cx: i32,
        cy: i32,
        flags: u32,
    ) -> i32;
    fn GetWindowLongW(hwnd: *mut std::ffi::c_void, n_index: i32) -> i32;
    fn SetWindowLongW(hwnd: *mut std::ffi::c_void, n_idx: i32, dw: i32) -> i32;
    fn GetAsyncKeyState(v_key: i32) -> i16;
    fn OpenClipboard(hwnd: *mut std::ffi::c_void) -> i32;
    fn CloseClipboard() -> i32;
    fn EmptyClipboard() -> i32;
    fn SetClipboardData(u_format: u32, h_mem: *mut std::ffi::c_void) -> *mut std::ffi::c_void;
    fn GetClipboardData(u_format: u32) -> *mut std::ffi::c_void;
    fn GetForegroundWindow() -> *mut std::ffi::c_void;
}
#![link(name = "kernel32")]
extern "system" {
    fn GlobalAlloc(u_flags: u32, dw_bytes: usize) -> *mut std::ffi::c_void;
    fn GlobalLock(h_mem: *mut std::ffi::c_void) -> *mut std::ffi::c_void;
    fn GlobalUnlock(h_mem: *mut std::ffi::c_void) -> i32;
    fn GlobalFree(h_mem: *mut std::ffi::c_void) -> *mut std::ffi::c_void;
}
const GWL_STYLE: i32 = -16;
const WS_CAPTION: i32 = 0x00C00000;
const WS_MAXIMIZEBOX: i32 = 0x00010000;
const WS_MINIMIZEBOX: i32 = 0x00020000;
const WS_THICKFRAME: i32 = 0x00040000;
const SWP_FRAMECHANGED: u32 = 0x0020;
const SWP_NOMOVE: u32 = 0x0002;
const SWP_NOSIZE: u32 = 0x0001;
const SWP_NOZORDER: u32 = 0x0004;
const CF_UNICODETEXT: u32 = 13;
const GMEM_MOVEABLE: u32 = 0x0002;
#[derive(Clone, Debug, Serialize, Deserialize)]
struct RawLink {
    id: i32,
    target_node_id: i32,
    target_node_name: String,
}
#[derive(Clone, Debug, Serialize, Deserialize)]
struct RawItem {
    id: i32,
    text: String,
    parent_id: i32,
    is_expanded: bool,
    is_editing: bool,
    note: String,
}

```

```

    links: Vec<RawLink>,
}
#[derive(Serialize, Deserialize)]
struct ClipboardBranch {
    root_item: RawItem,
    children: Vec<ClipboardBranch>,
}
#[derive(Clone, Copy)]
struct Point2D {
    x: f32,
    y: f32,
}
#[derive(Clone, Debug, Serialize, Deserialize)]
struct CADTree {
    id: i32,
    cad_x: f32,
    cad_y: f32,
    width: f32,
    count: i32,
    items: Vec<RawItem>,
}
struct AppState {
    trees: Vec<CADTree>,
    cad_links: Vec<CADLine>,
    selected_node_id: i32,
    active_focus_tree_id: i32,
    history: Vec<Vec<CADTree>>,
}
fn set_clipboard_text(text: &str) {
    unsafe {
        if OpenClipboard(std::ptr::null_mut()) != 0 {
            EmptyClipboard();
            let v: Vec<u16> = text.encode_utf16().chain(std::iter::once(0)).collect();
            let h_mem = GlobalAlloc(GMEM_MOVEABLE, v.len() * 2);
            if !h_mem.is_null() {
                let ptr = GlobalLock(h_mem) as *mut u16;
                std::ptr::copy_nonoverlapping(v.as_ptr(), ptr, v.len());
                GlobalUnlock(h_mem);
                if SetClipboardData(CF_UNICODETEXT, h_mem).is_null() {
                    GlobalFree(h_mem);
                }
            }
            CloseClipboard();
        }
    }
}
fn get_clipboard_text() -> Option<String> {
    unsafe {
        if OpenClipboard(std::ptr::null_mut()) != 0 {
            let h_mem = GetClipboardData(CF_UNICODETEXT);
            if !h_mem.is_null() {
                let ptr = GlobalLock(h_mem) as *const u16;
                if !ptr.is_null() {
                    let mut len = 0;
                    while *ptr.add(len) != 0 {
                        len += 1;
                    }
                    let slice = std::slice::from_raw_parts(ptr, len);
                    let res = String::from_utf16(slice).ok();
                    GlobalUnlock(h_mem);
                    CloseClipboard();
                }
            }
        }
    }
}

```

```

        return res;
    }
    GlobalUnlock(h_mem);
}
CloseClipboard();
}
}
None
}
fn check_collision(x1: f32, y1: f32, w1: f32, h1: f32, x2: f32, y2: f32, w2: f32, h2: f32) -> bool {
    let pad = 15.0_f32;
    x1 < x2 + w2 + pad && x1 + w1 + pad > x2 && y1 < y2 + h2 + pad && y1 + h1 + pad > y2
}
// ИСПРАВЛЕНО: Секретная величина теперь рассчитывается как 10 * на сумму ВСЕХ узлов во ВСЕХ
деревьях на сцене
fn find_closest_free_pos(
    moved_id: i32,
    mx: f32,
    my: f32,
    mw: f32,
    mh: f32,
    trees: &[CADTree],
) -> (f32, f32) {
    let grid = 50.0_f32;
    let mut out_x = (mx / grid).round() * grid;
    let mut out_y = (my / grid).round() * grid;
    let total_all_nodes: usize = trees.iter().map(|t| t.items.len()).sum();
    let secret_value = (total_all_nodes as f32) * 10.0;
    for t in trees {
        if t.id == moved_id {
            continue;
        }
        let th = (t.count * 32 + 16) as f32;
        if check_collision(out_x, out_y, mw, mh, t.cad_x, t.cad_y, t.width, th) {
            let delta_x = (mw / 2.0) + (t.width / 2.0) + secret_value;
            let grid_delta_x = (delta_x / grid).round() * grid;
            if out_x + (mw / 2.0) < t.cad_x + (t.width / 2.0) {
                out_x = t.cad_x - grid_delta_x;
            } else {
                out_x = t.cad_x + grid_delta_x;
            }
            out_y = (out_y / grid).round() * grid;
        }
    }
    (out_x, out_y)
}
fn build_tree(
    pid: i32,
    depth: i32,
    db: &Vec<RawItem>,
    v_items: &mut Vec<MyTreeItem>,
    hidden: bool,
    parent_last: Vec<bool>,
    sel_id: i32,
    _parent_coord: Point2D,
    global_index: &mut i32,
) {
    let mut children: Vec<&RawItem> = db.iter().filter(|i| i.parent_id == pid).collect();
    children.sort_by_key(|i| i.id);
    let total_children = children.len();
    for (index, item) in children.into_iter().enumerate() {

```

```

let is_last = index == total_children - 1;
let has_children = db.iter().any(|i| i.parent_id == item.id);
let current_row_idx = *global_index;
if !hidden {
    *global_index += 1;
}
let current_node_coord = Point2D {
    x: 8.0 + 10.0 + (depth as f32 * 24.0) + 10.0,
    y: 8.0 + (current_row_idx as f32 * 32.0) + 16.0,
};
if !hidden {
    let computed_w = (item.text.chars().count() as f32 * 7.2) + 20.0;
    let smart_id = format!(
        "{}_{}_{}_{}_{}",
        item.parent_id,
        item.id,
        depth,
        if is_last { "L" } else { "M" },
        item.links.len()
    );
    let mut slint_mask = [0; 12];
    for i in 0..12 {
        if i < (depth - 1) as usize && i < parent_last.len() {
            slint_mask[i] = if !parent_last[i] { 1 } else { 0 };
        }
    }
    v_items.push(MyTreeItem {
        id: item.id,
        text: item.text.clone().into(),
        depth,
        is_last,
        has_children,
        is_editing: item.is_editing,
        is_expanded: item.is_expanded,
        parent_is_last: depth > 1 && parent_last[(depth - 2) as usize],
        is_selected: item.id == sel_id,
        note: item.note.clone().into(),
        text_width: computed_w,
        smart_id: smart_id.into(),
        line_mask: ModelRc::from(Rc::new(VecModel::from(slint_mask.to_vec()))),
        parent_id: item.parent_id,
        parent_triangle_x: _parent_coord.x,
        parent_triangle_y: _parent_coord.y,
        self_triangle_x: current_node_coord.x,
        self_triangle_y: current_node_coord.y,
    });
}
if has_children {
    let mut current_last = parent_last.clone();
    current_last.push(is_last);
    build_tree(
        item.id,
        depth + 1,
        db,
        v_items,
        hidden || !item.is_expanded,
        current_last,
        sel_id,
        current_node_coord,
        global_index,
    );
}

```

```

    }
}
}
fn update_ui_models(app: &AppWindow, state: &AppState) {
    let mut slint_trees = Vec::new();
    for t in &state.trees {
        let mut t_views = Vec::new();
        let mut g_idx = 0;
        build_tree(
            0,
            0,
            &t.items,
            &mut t_views,
            false,
            vec![],
            state.selected_node_id,
            Point2D { x: 20.0, y: 24.0 },
            &mut g_idx,
        );
        let mut max_w = 250.0_f32;
        for item in &t_views {
            max_w = max_w.max(10.0 + (item.depth as f32 * 24.0) + 20.0 + item.text_width + 40.0);
        }
        slint_trees.push(SlintCADTree {
            id: t.id,
            cad_x: t.cad_x,
            cad_y: t.cad_y,
            width: max_w,
            count: g_idx,
            data: ModelRc::from(Rc::new(VecModel::from(t_views))),
        });
    }
    app.set_trees_list(ModelRc::from(Rc::new(VecModel::from(slint_trees))));
    let mut updated_links = Vec::new();
    for link in &state.cad_links {
        let mut line = link.clone();
        let (mut f_src, mut f_dst) = (false, false);
        for t in &state.trees {
            let mut t_views = Vec::new();
            let mut g_idx = 0;
            build_tree(
                0,
                0,
                &t.items,
                &mut t_views,
                true,
                vec![],
                0,
                Point2D { x: 0.0, y: 0.0 },
                &mut g_idx,
            );
            if let Some(item) = t_views.iter().find(|i| i.id == link.id_obj1) {
                line.x1 = t.cad_x + item.self_triangle_x;
                line.y1 = t.cad_y + item.self_triangle_y;
                f_src = true;
            }
            if let Some(item) = t_views.iter().find(|i| i.id == link.id_obj2) {
                line.x2 = t.cad_x + item.self_triangle_x;
                line.y2 = t.cad_y + item.self_triangle_y;
                f_dst = true;
            }
        }
    }
}

```

```

    }
    if f_src && f_dst {
        updated_links.push(line);
    }
}
app.set_active_cad_links(ModelRc::from(Rc::new(VecModel::from(updated_links))));
}
fn pack_branch(pid: i32, vec: &Vec<RawItem>) -> Vec<ClipboardBranch> {
    vec.iter()
        .filter(|i| i.parent_id == pid)
        .map(|i| ClipboardBranch {
            root_item: i.clone(),
            children: pack_branch(i.id, vec),
        })
        .collect()
}
fn unpack_branch(branch: ClipboardBranch, new_parent_id: i32, vec: &mut Vec<RawItem>) {
    let next_id = vec.iter().map(|i| i.id).max().unwrap_or(0) + 1;
    let mut new_item = branch.root_item;
    new_item.id = next_id;
    new_item.parent_id = new_parent_id;
    new_item.is_editing = false;
    vec.push(new_item);
    for child in branch.children {
        unpack_branch(child, next_id, vec);
    }
}
}
fn main() {
    let app = AppWindow::new().unwrap();
    let app_weak = app.as_weak();
    let state = Rc::new(RefCell::new(AppState {
        trees: vec![CADTree {
            id: 1,
            cad_x: 40.0,
            cad_y: 80.0,
            width: 250.0,
            count: 1,
            items: vec![RawItem {
                id: 1,
                text: "Основной узел".into(),
                parent_id: 0,
                is_expanded: true,
                is_editing: false,
                note: "Главный server управления".into(),
                links: vec![],
            }],
        }],
        cad_links: Vec::new(),
        selected_node_id: 1,
        active_focus_tree_id: 1,
        history: Vec::new(),
    }));
    update_ui_models(&app, &state.borrow());
    let _s = state.clone();
    let a_w = app_weak.clone();
    app.on_toggle_expand(move |node_id, tree_id| {
        let mut st = _s.borrow_mut();
        if let Some(t) = st.trees.iter_mut().find(|t| t.id == tree_id) {
            if let Some(i) = t.items.iter_mut().find(|i| i.id == node_id) {
                i.is_expanded = !i.is_expanded;
            }
        }
    })
}

```

```

    }
    update_ui_models(&a_w.unwrap(), &st);
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_trees_geometry_changed(move || {
    update_ui_models(&a_w.unwrap(), &_s.borrow());
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_update_tree_position(move |tree_id, nx, ny| {
    let mut st = _s.borrow_mut();
    if let Some(t) = st.trees.iter_mut().find(|t| t.id == tree_id) {
        t.cad_x = nx;
        t.cad_y = ny;
    }
    update_ui_models(&a_w.unwrap(), &st);
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_resolve_tree_collision(move |tree_id| {
    let ui = a_w.unwrap();
    let mut st = _s.borrow_mut();
    let (mut cur_x, mut cur_y, mut mw, mut mh) = (0.0, 0.0, 250.0, 48.0);
    if let Some(t) = st.trees.iter().find(|t| t.id == tree_id) {
        cur_x = t.cad_x;
        cur_y = t.cad_y;
        mw = t.width;
        mh = (t.count * 32 + 16) as f32;
    }
    let (fx, fy) = find_closest_free_pos(tree_id, cur_x, cur_y, mw, mh, &st.trees);
    if let Some(t) = st.trees.iter_mut().find(|t| t.id == tree_id) {
        t.cad_x = fx;
        t.cad_y = fy;
    }
    update_ui_models(&ui, &st);
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_add_main_tree(move |win_w, win_h| {
    let ui = a_w.unwrap();
    let mut st = _s.borrow_mut();
    let n_id = st.trees.iter().map(|t| t.id).max().unwrap_or(0) + 1;
    let target_x = (win_w / 2.0) - 125.0;
    let target_y = (win_h / 2.0) - 24.0;
    let (fx, fy) = find_closest_free_pos(n_id, target_x, target_y, 250.0, 48.0, &st.trees);
    st.trees.push(CADTree {
        id: n_id,
        cad_x: fx,
        cad_y: fy,
        width: 250.0,
        count: 1,
        items: vec![RawItem {
            id: n_id * 1000,
            text: format!("Основной узел {}", n_id),
            parent_id: 0,
            is_expanded: true,
            is_editing: false,
            note: "".into(),
            links: vec![],
        }],
    });
});

```

```

});
update_ui_models(&ui, &st);
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_hover_item(move |node_id, idx, tree_id| {
    let ui = a_w.unwrap();
    let st = _s.borrow();
    if node_id != -1 {
        if let Some(t) = st.trees.iter().find(|t| t.id == tree_id) {
            if let Some(i) = t.items.iter().find(|i| i.id == node_id) {
                ui.set_tooltip_text(
                    (if i.note.is_empty() {
                        "Примечание отсутствует."
                    } else {
                        &i.note
                    })
                    .into(),
                );
                ui.set_tooltip_index(idx);
            }
        }
    } else {
        ui.set_tooltip_index(-1);
    }
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_select_item(move |node_id, _txt, tree_id| {
    let ui = a_w.unwrap();
    let mut st = _s.borrow_mut();
    st.active_focus_tree_id = tree_id;
    st.selected_node_id = node_id;
    if let Some(t) = st.trees.iter().find(|t| t.id == tree_id) {
        if let Some(i) = t.items.iter().find(|i| i.id == node_id) {
            ui.set_properties_text(i.note.clone().into());
        }
    }
    update_ui_models(&ui, &st);
});
let _s = state.clone();
app.on_change_note(move |node_id, note, tree_id| {
    let mut st = _s.borrow_mut();
    if let Some(t) = st.trees.iter_mut().find(|t| t.id == tree_id) {
        if let Some(i) = t.items.iter_mut().find(|i| i.id == node_id) {
            i.note = note.to_string();
        }
    }
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_item_double_clicked(move |node_id, tree_id| {
    let ui = a_w.unwrap();
    let mut st = _s.borrow_mut();
    if let Some(t) = st.trees.iter_mut().find(|t| t.id == tree_id) {
        for i in t.items.iter_mut() {
            i.is_editing = i.id == node_id;
        }
    }
    update_ui_models(&ui, &st);
});

```

```

let _s = state.clone();
let a_w = app_weak.clone();
app.on_finish_rename(move |node_id, txt, tree_id| {
    let ui = a_w.unwrap();
    let mut st = _s.borrow_mut();
    if let Some(t) = st.trees.iter_mut().find(|t| t.id == tree_id) {
        if let Some(i) = t.items.iter_mut().find(|i| i.id == node_id) {
            i.text = txt.to_string();
            i.is_editing = false;
        }
    }
    update_ui_models(&ui, &st);
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_menu_action(move |act, n_id, tree_id| {
    let ui = a_w.unwrap();
    let mut st = _s.borrow_mut();
    let max_node_id = st
        .trees
        .iter()
        .flat_map(|t| t.items.iter().map(|i| i.id))
        .max()
        .unwrap_or(0)
        + 1;
    if let Some(t) = st.trees.iter_mut().find(|t| t.id == tree_id) {
        match act.as_str() {
            "add" => {
                if let Some(p) = t.items.iter_mut().find(|i| i.id == n_id) {
                    p.is_expanded = true;
                }
                t.items.push(RawItem {
                    id: max_node_id,
                    text: format!("Новый узел {}", max_node_id),
                    parent_id: n_id,
                    is_expanded: true,
                    is_editing: true,
                    note: "".into(),
                    links: vec![],
                });
                st.selected_node_id = max_node_id;
            }
            "delete" => {
                fn del(pid: i32, vec: &mut Vec<RawItem>) {
                    let sub: Vec<i32> = vec
                        .iter()
                        .filter(|i| i.parent_id == pid)
                        .map(|i| i.id)
                        .collect();
                    for s in sub {
                        del(s, vec);
                    }
                    vec.retain(|i| i.id != pid);
                }
                if n_id % 1000 != 0 {
                    del(n_id, &mut t.items);
                }
            }
            "rename" => {
                if let Some(i) = t.items.iter_mut().find(|i| i.id == n_id) {
                    i.is_editing = true;
                }
            }
        }
    }
});

```

```

    }
  }
  "properties" => {
    if let Some(i) = t.items.iter().find(|i| i.id == n_id) {
      ui.set_properties_text(i.note.clone().into());
      ui.set_properties_visible(true);
    }
  }
  _ => {}
}
}
update_ui_models(&ui, &st);
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_undo_action(move || {
  let mut st = _s.borrow_mut();
  if st.active_focus_tree_id == 1 {
    if let Some(p) = st.history.pop() {
      st.trees = p;
    }
  }
  update_ui_models(&a_w.unwrap(), &st);
});
let _s = state.clone();
app.on_copy_action(move || {
  let st = _s.borrow();
  let tree_id = st.active_focus_tree_id;
  if let Some(t) = st.trees.iter().find(|t| t.id == tree_id) {
    let sel = st.selected_node_id;
    if let Some(i) = t.items.iter().find(|i| i.id == sel) {
      if i.is_editing {
        set_clipboard_text(&i.text);
      } else {
        let branch = ClipboardBranch {
          root_item: i.clone(),
          children: pack_branch(i.id, &t.items),
        };
        if let Ok(j) = serde_json::to_string(&branch) {
          set_clipboard_text(&format!("TREE_BRANCH_DATA:{}", j));
        }
      }
    }
  }
});
let _s = state.clone();
let a_w = app_weak.clone();
app.on_paste_action(move || {
  if let Some(txt) = get_clipboard_text() {
    let (tree_id, sel, is_edit, old) = {
      let st = _s.borrow();
      (
        st.active_focus_tree_id,
        st.selected_node_id,
        st.trees
          .iter()
          .flat_map(|t| t.items.iter())
          .find(|i| i.id == st.selected_node_id)
          .map(|i| i.is_editing)
          .unwrap_or(false),
        st.trees.clone(),
      )
    }
  }
});

```

```

    )
};
let mut st = _s.borrow_mut();
if is_edit {
    if let Some(t) = st.trees.iter_mut().find(|t| t.id == tree_id) {
        if let Some(i) = t.items.iter_mut().find(|i| i.id == sel) {
            i.text = txt;
        }
    }
} else if txt.starts_with("TREE_BRANCH_DATA:") {
    st.history.push(old);
    let json = txt.chars().skip(17).collect::<String>();
    if let Ok(br) = serde_json::from_str::<ClipboardBranch>(&json) {
        if let Some(t) = st.trees.iter_mut().find(|t| t.id == tree_id) {
            unpack_branch(br, sel, &mut t.items);
            if let Some(p) = t.items.iter_mut().find(|i| i.id == sel) {
                p.is_expanded = true;
            }
        }
    }
}
update_ui_models(&a_w.unwrap(), &st);
});
let app_win = app.as_weak();
slint::invoke_from_event_loop(move || {
    if let Some(_ui) = app_win.upgrade() {
        let title: Vec<u16> = "Менеджер команд голосового управления (CAD Среда)\0"
            .encode_utf16()
            .collect();
        unsafe {
            let hwnd = FindWindowW(std::ptr::null(), title.as_ptr());
            if !hwnd.is_null() {
                let cur = GetWindowLongW(hwnd, GWL_STYLE);
                SetWindowLongW(
                    hwnd,
                    GWL_STYLE,
                    cur | WS_CAPTION | WS_MINIMIZEBOX | WS_MAXIMIZEBOX | WS_THICKFRAME,
                );
                SetWindowPos(
                    hwnd,
                    std::ptr::null_mut(),
                    0,
                    0,
                    0,
                    0,
                    SWP_FRAMECHANGED | SWP_NOMOVE | SWP_NOSIZE | SWP_NOZORDER,
                );
                ShowWindow(hwnd, 3);
            }
        }
    }
})
.unwrap();
spawn_hotkey_thread(app.as_weak());
app.run().unwrap();
}
fn spawn_hotkey_thread(app_weak: Weak<AppWindow>) {
    thread::spawn(move || {
        let (mut c_w, mut z_w, mut cc_w, mut v_w) = (false, false, false, false);
        let title: Vec<u16> = "Менеджер команд голосового управления (CAD Среда)\0"

```

```
.encode_utf16()
.collect();
let mut hw: *mut std::ffi::c_void = std::ptr::null_mut();
loop {
    thread::sleep(Duration::from_millis(50));
    unsafe {
        if hw.is_null() {
            hw = FindWindowW(std::ptr::null(), title.as_ptr());
        }
        let active = GetForegroundWindow();
        if !hw.is_null() && (active == hw) {
            let ctrl = (GetAsyncKeyState(0x11) & -32768_i16) != 0;
            let z = (GetAsyncKeyState(0x5A) & -32768_i16) != 0;
            let c = (GetAsyncKeyState(0x43) & -32768_i16) != 0;
            let v = (GetAsyncKeyState(0x56) & -32768_i16) != 0;
            if ctrl && z && (!c_w || !z_w) {
                let aw = app_weak.clone();
                let _ = slint::invoke_from_event_loop(move || {
                    if let Some(a) = aw.upgrade() {
                        a.invoke_undo_action();
                    }
                });
            }
            if ctrl && c && (!c_w || !cc_w) {
                let aw = app_weak.clone();
                let _ = slint::invoke_from_event_loop(move || {
                    if let Some(a) = aw.upgrade() {
                        a.invoke_copy_action();
                    }
                });
            }
            if ctrl && v && (!c_w || !v_w) {
                let aw = app_weak.clone();
                let _ = slint::invoke_from_event_loop(move || {
                    if let Some(a) = aw.upgrade() {
                        a.invoke_paste_action();
                    }
                });
            }
            c_w = ctrl;
            z_w = z;
            cc_w = c;
            v_w = v;
        }
    }
}
});
```